

1 Деревья и отрезки

1.1 Домашнее задание

1. Дана последовательность чисел длины n . Придумайте как за $\mathcal{O}(n \log n + m\sqrt{n} \log n)$ ответить на m следующих запросов:
 - `sum(l, r)` – посчитать сумму на отрезке $[l, r]$.
 - `inc(l, r, x)` – увеличить все значения отрезка $[l, r]$, меньшие x , до x .
2. Дана последовательность длины n . Элементы последовательности — целые числа, не превосходящие C . Научитесь обрабатывать online-запросы:
 - Циклически сдвинуть подотрезок последовательности на 1 влево или вправо. Например, в последовательности $(1, 2, 3, 4, 5)$ циклический сдвиг $(2, 3, 4)$ на 1 вправо даст последовательность $(1, 4, 2, 3, 5)$
 - Посчитать, сколько раз встречается число x на подотрезке последовательности. $\mathcal{O}(\sqrt{n})$ на запрос, предподсчёт за $\mathcal{O}(C\sqrt{n} + n)$.
3. k -инверсией в перестановке p называется набор индексов $i_1 < i_2 < \dots < i_k$, такой, что $p[i_1] > p[i_2] > \dots > p[i_k]$. Найти число k -инверсий за $\mathcal{O}(nk \log n)$.
4. Дана последовательность из n чисел и m отрезков $[l_i, r_i]$ на ней. Про каждый отрезок надо определить число инверсий на нем. $\mathcal{O}((m+n)\sqrt{n} \log n)$.

Дополнительные задачи

5. Даны натуральные числа $a_1, \dots, a_n$ и k , не превосходящие C . За $\mathcal{O}(C + (m+n)\sqrt{n})$ ответить на m запросов вида: по данным l, r найти количество таких i и j , что $l \leq i \leq j \leq r$ и `xor` чисел $a_i, a_{i+1}, \dots, a_j$ равен k .

1.2 Решение

1. Разобьём последовательность на блоки по $\sqrt{n}$ элементов. Для каждого блока будем хранить массив префиксных сумм и отсортированный список всех элементов блока.

Так как мы сортируем $\sqrt{n}$ элементов в $\sqrt{n}$ блоках, то сложность такой операции составит $O(n \log n)$, а префиксные суммы считаются за $O(n)$

Операция увеличения элементов на отрезке $[l, r]$

Отрезок $[l, r]$ может покрывать полные и частичные блоки.

- Для **полных** блоков выполним ленивое обновление — сохраним $\max(T)$, где T — элемент, до которого нужно увеличить элементы меньшие T внутри блока.
- Для **частичных** блоков явно пройдемся по элементам, обновим покрываемые элементы, пересчитаем суммы. Также учтем потенциальный T , если блок до этого был обработан как полный.

Операция суммы на отрезке $[l, r]$

Аналогично предыдущему случаю:

- Для **частичных** блоков пройдемся по элементам: если $x_i < T$, то возьмём T вместо него, иначе сам x_i .
- Для **полных** блоков найдём первый элемент $x_i \geq T$ и на основе префиксной суммы и отсортированного списка вычислим сумму отрезка как

$$S = \sum_{x_i \geq T} x_i + T \cdot |\{x_i \mid x_i < T\}|,$$

где

- $\sum_{x_i \geq T} x_i$ — сумма элементов, больших или равных T (посчитана в префиксной сумме),
- $|\{x_i \mid x_i < T\}|$ — количество элементов, меньших T .

Сложность обеих операций составляет $O(m\sqrt{n} \log n)$, так как мы рассматриваем $O(\sqrt{n})$ блоков и выполняем в каждом бинарный поиск $O(\log n)$.

2. Разобьём последовательность на блоки по $\sqrt{n}$ элементов. Для каждого блока будем хранить *deque* с элементами блока и *map* с количеством вхождений каждого элемента.

Операция циклического сдвига

Рассмотрим сдвиг вправо, для сдвига влево аналогично, но в обратную сторону.

Каждый блок достаёт последний элемент из своего *deque* и добавляет его в начало *deque* следующего блока. Последний блок добавляет элемент в начало первого блока на отрезке.

Также в момент сдвига уменьшаем/увеличиваем количество вхождений элементов в *map*.

Операция подсчета количества вхождений

Отрезок $[l, r]$ может покрывать полные и частичные блоки.

- Для **полных** блоков просто добавляем количество вхождений каждого элемента блока в ответ из *map*.
- Для **частичных** блоков явно пройдемся по элементам, добавим количество вхождений в ответ.

Сложность обеих операций составляет $O(\sqrt{n})$, рассматриваем $O(\sqrt{n})$ блоков и выполняем в каждой операции за $O(1)$. На преподсчет тратим время на разбиение последовательности на блоки и подсчет количества вхождений элементов в блоке — $O(C\sqrt{n} + n)$.

3. Определим $dp[i][j]$ — количество убывающих подпоследовательностей длины j , заканчивающихся в позиции i . Тогда:

- База: $dp[i][1] = 1$ для всех i (каждый элемент сам по себе является подпоследовательностью длины 1).
- Переход:

$$dp[i][j] = \sum_{t < i, p[t] > p[i]} dp[t][j-1].$$

Суммируем все возможные $(j-1)$ -подпоследовательности, заканчивающиеся в t , где $p[t] > p[i]$.

Прямой подсчёт суммы в рекуррентной формуле занял бы $O(n)$ на каждый $dp[i][j]$, что даёт $O(n^2k)$ в худшем случае. Чтобы ускорить вычисления, будем использовать **дерево отрезков**.

- Заводим массив деревьев отрезков $seg[j]$, где $seg[j]$ позволяет получать суммы значений $dp[*][j]$.
- В каждом узле дерева отрезков храним сумму значений $dp[i][j]$ для соответствующего диапазона индексов.
- Чтобы вычислить $dp[i][j]$, находим сумму на отрезке $[p[i] + 1, n]$ с помощью дерева $seg[j-1]$.
- После вычисления $dp[i][j]$ обновляем дерево $seg[j]$ по индексу $p[i]$.

Получаем сложность $O(nk \log n)$.

4. Устал и лень писать решение :)

Надо поиграться с алгоритмом Мо.